

08 - Material - complexidade de algoritmos - melhor

May 10, 2026

Complexidade de Algoritmos (Big-O) com exemplos em Linguagem C

Objetivo: entender como analisar a eficiência de algoritmos e comparar o crescimento do custo de execução e de memória conforme o tamanho da entrada aumenta.

Este material foi adaptado para uma disciplina de **Estrutura de Dados em Linguagem C**. Os exemplos de algoritmos estão em C, enquanto os gráficos continuam sendo gerados em Python para facilitar a visualização em sala de aula.

1 Motivação: por que complexidade importa?

Em Estrutura de Dados, quase sempre existem várias formas de resolver o mesmo problema. A diferença entre elas não está apenas em “funcionar”, mas em **como o custo cresce** quando a quantidade de dados aumenta.

Por exemplo:

- procurar um número em um vetor pequeno pode parecer rápido de qualquer forma;
- procurar em um vetor com 1 milhão de posições já exige mais cuidado;
- ordenar poucos valores é simples, mas ordenar milhares ou milhões de registros pode ficar muito lento dependendo do algoritmo.

A análise de complexidade ajuda a responder perguntas como:

- Quantas operações o algoritmo tende a executar?
- O tempo cresce pouco ou cresce muito quando **n** aumenta?
- O algoritmo usa memória extra?
- A estrutura de dados escolhida ajuda ou atrapalha?

2 Notações: O , Ω e Θ

As principais notações usadas para análise assintótica são:

- **$O(g(n))$** : limite superior. Geralmente usado para representar o pior caso ou uma estimativa máxima de crescimento.
- **$\Omega(g(n))$** : limite inferior. Representa o melhor caso ou uma estimativa mínima de crescimento.
- **$\Theta(g(n))$** : limite apertado. Usado quando o algoritmo cresce exatamente naquela ordem, tanto superior quanto inferiormente.

Na prática, em disciplinas introdutórias, usamos mais o **Big-O**, porque ele ajuda a estimar como o algoritmo se comporta nos cenários mais críticos.

Exemplo: dizer que uma busca linear é **$O(n)$** significa que, no pior caso, talvez seja necessário percorrer todos os n elementos.

3 Tempo e espaço em C

Em C, é importante analisar dois custos:

3.1 Tempo

Refere-se à quantidade de operações executadas, como:

- comparações;
- atribuições;
- incrementos;
- chamadas de função;
- acessos a posições de vetor;
- laços de repetição.

3.2 Espaço

Refere-se à memória usada pelo algoritmo.

Em C, isso é especialmente importante porque o programador lida diretamente com:

- vetores estáticos;
- vetores alocados dinamicamente com `malloc()`;
- liberação de memória com `free()`;
- ponteiros;
- estruturas (`struct`);
- listas encadeadas.

Um algoritmo pode ser rápido, mas consumir muita memória. Outro pode usar pouca memória, mas ser lento. A escolha depende do problema.

4 Classes de complexidade

As classes mais comuns são:

- **$O(1)$ — constante:** o custo não depende de n . Exemplo: acessar `vetor[3]`.
- **$O(\log n)$ — logarítmica:** o problema é reduzido pela metade a cada etapa. Exemplo: busca binária.
- **$O(n)$ — linear:** percorre todos os elementos uma vez. Exemplo: busca linear.
- **$O(n \log n)$ — quase linear:** comum em algoritmos eficientes de ordenação, como Merge Sort e Quick Sort no caso médio.
- **$O(n^2)$ — quadrática:** geralmente ocorre com dois laços aninhados. Exemplo: Bubble Sort.
- **$O(2^n)$ e $O(n!)$ — explosivas:** crescem muito rapidamente. Aparecem em problemas de força bruta, combinações e permutações.

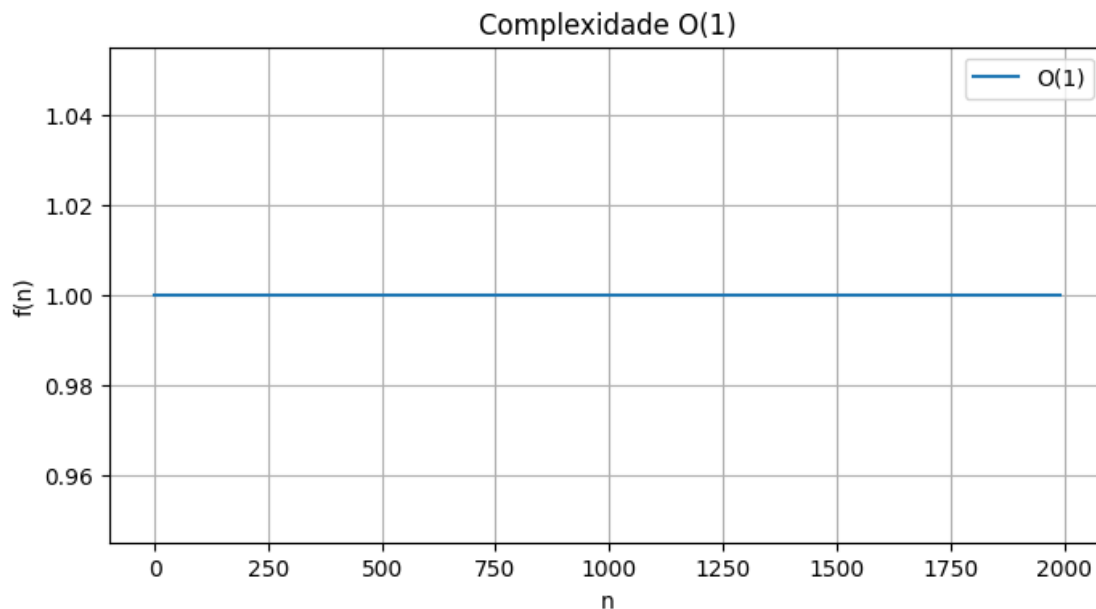
5 Gráficos individuais de crescimento

Os gráficos a seguir são gerados em Python. Eles não fazem parte da implementação dos algoritmos em C; servem para visualizar como cada função cresce conforme n aumenta.

```
[1]: # Gráfico individual:  $O(1)$ 
import matplotlib.pyplot as plt

ns = list(range(1, 2001, 10))
o1 = [1 for _ in ns]

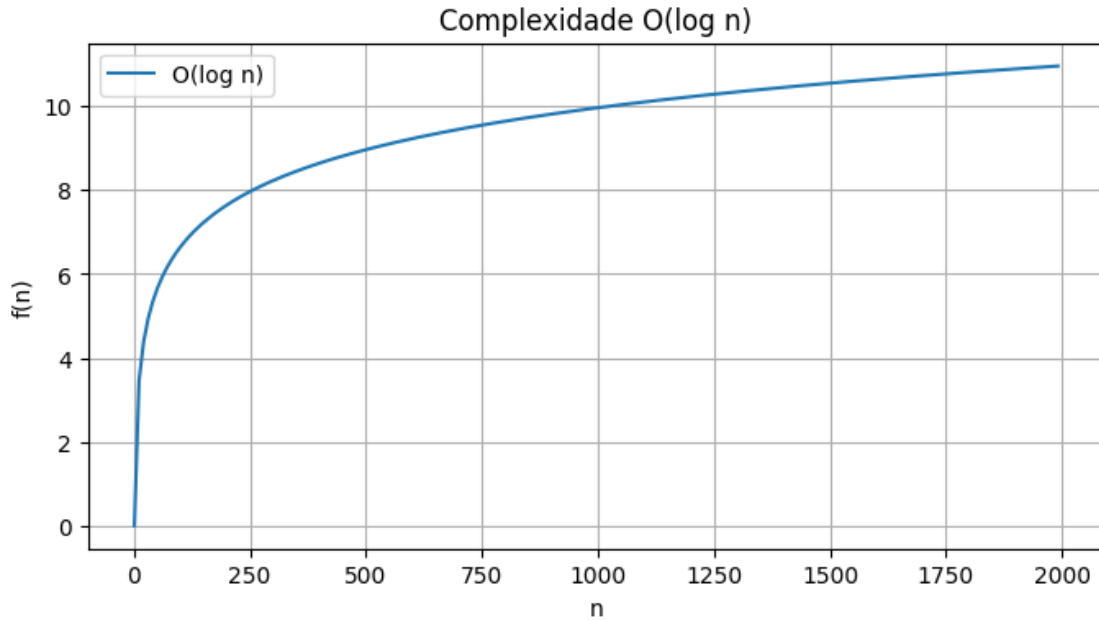
plt.figure(figsize=(8, 4))
plt.plot(ns, o1, label=" $O(1)$ ")
plt.title("Complexidade  $O(1)$ ")
plt.xlabel("n")
plt.ylabel("f(n)")
plt.grid(True)
plt.legend()
plt.show()
```



```
[2]: # Gráfico individual:  $O(\log n)$ 
import math
import matplotlib.pyplot as plt

ns = list(range(1, 2001, 10))
ologn = [math.log2(n) for n in ns]
```

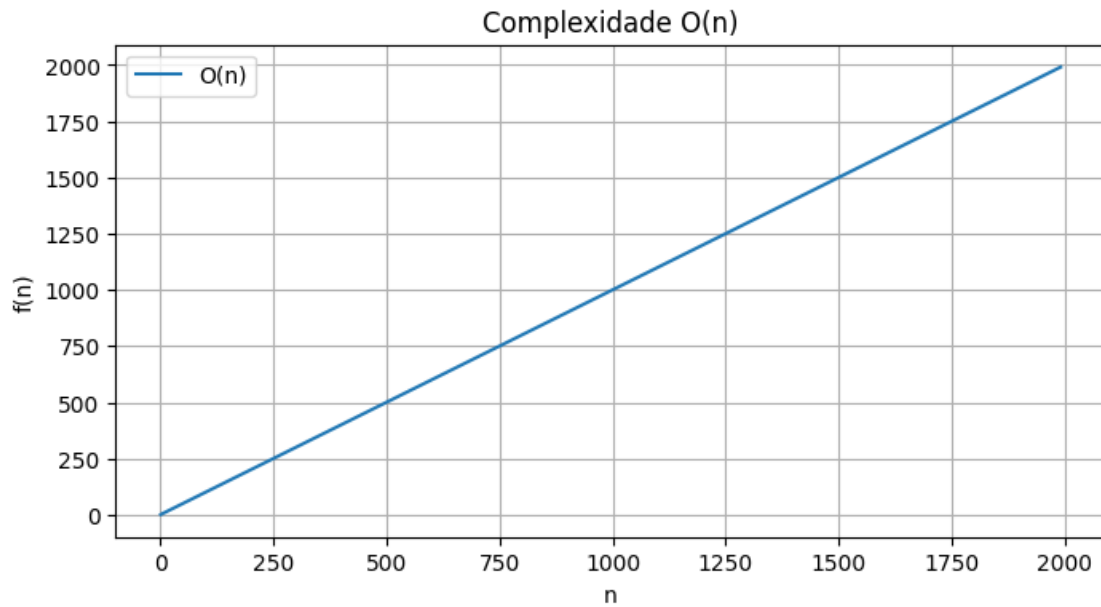
```
plt.figure(figsize=(8, 4))
plt.plot(ns, ologn, label="O(log n)")
plt.title("Complexidade O(log n)")
plt.xlabel("n")
plt.ylabel("f(n)")
plt.grid(True)
plt.legend()
plt.show()
```



```
[3]: # Gráfico individual: O(n)
import matplotlib.pyplot as plt

ns = list(range(1, 2001, 10))
on = [n for n in ns]

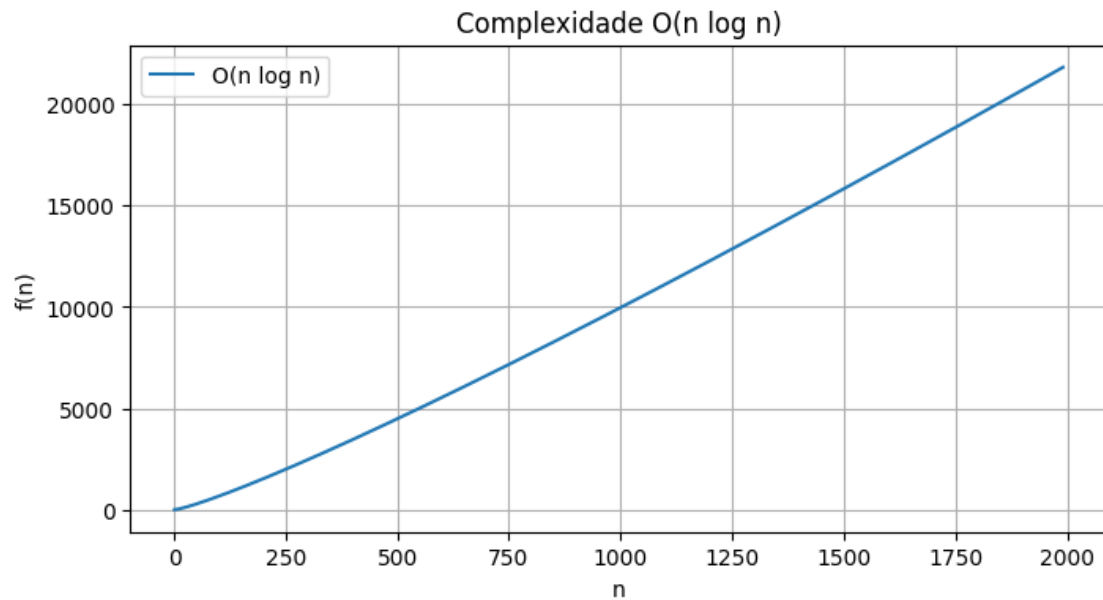
plt.figure(figsize=(8, 4))
plt.plot(ns, on, label="O(n)")
plt.title("Complexidade O(n)")
plt.xlabel("n")
plt.ylabel("f(n)")
plt.grid(True)
plt.legend()
plt.show()
```



```
[4]: # Gráfico individual:  $O(n \log n)$ 
import math
import matplotlib.pyplot as plt

ns = list(range(1, 2001, 10))
onlogn = [n * math.log2(n) for n in ns]

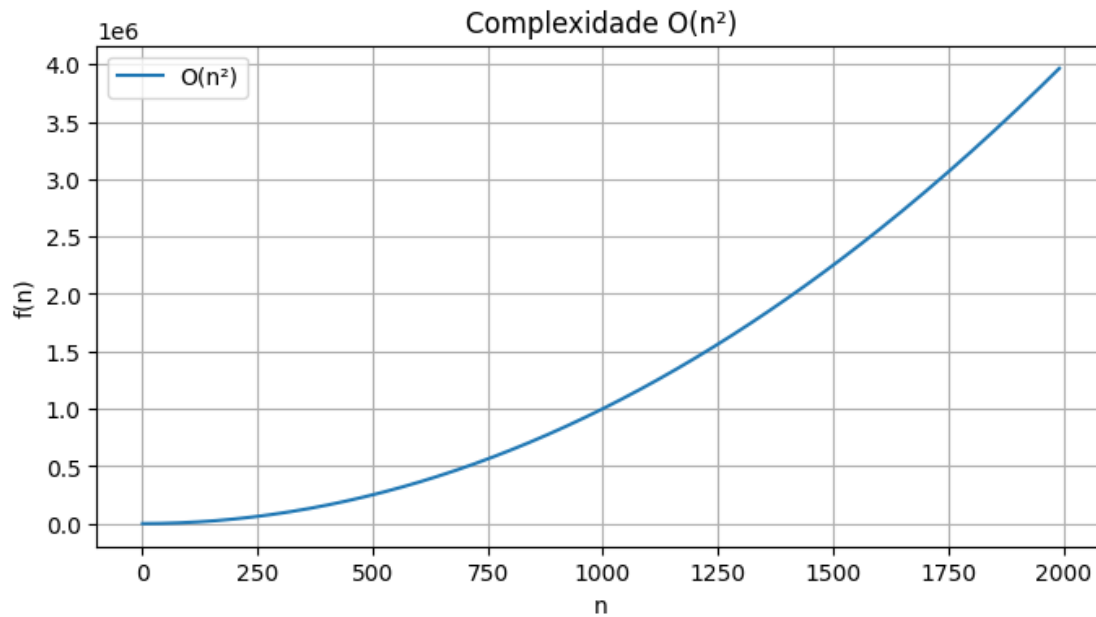
plt.figure(figsize=(8, 4))
plt.plot(ns, onlogn, label=" $O(n \log n)$ ")
plt.title("Complexidade  $O(n \log n)$ ")
plt.xlabel("n")
plt.ylabel("f(n)")
plt.grid(True)
plt.legend()
plt.show()
```



```
[5]: # Gráfico individual:  $O(n^2)$ 
import matplotlib.pyplot as plt

ns = list(range(1, 2001, 10))
on2 = [n * n for n in ns]

plt.figure(figsize=(8, 4))
plt.plot(ns, on2, label=" $O(n^2)$ ")
plt.title("Complexidade  $O(n^2)$ ")
plt.xlabel("n")
plt.ylabel("f(n)")
plt.grid(True)
plt.legend()
plt.show()
```

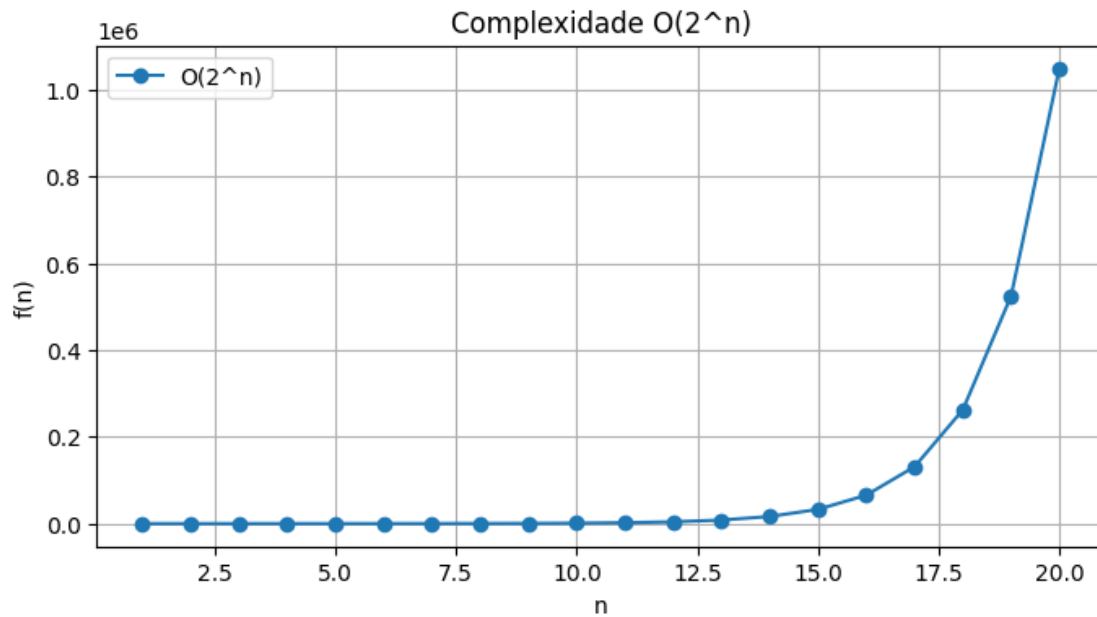


Para complexidades exponenciais e fatoriais, o valor de n precisa ser bem menor, porque o crescimento é muito rápido.

```
[6]: # Gráfico individual:  $O(2^n)$ 
import matplotlib.pyplot as plt

ns = list(range(1, 21))
o2n = [2**n for n in ns]

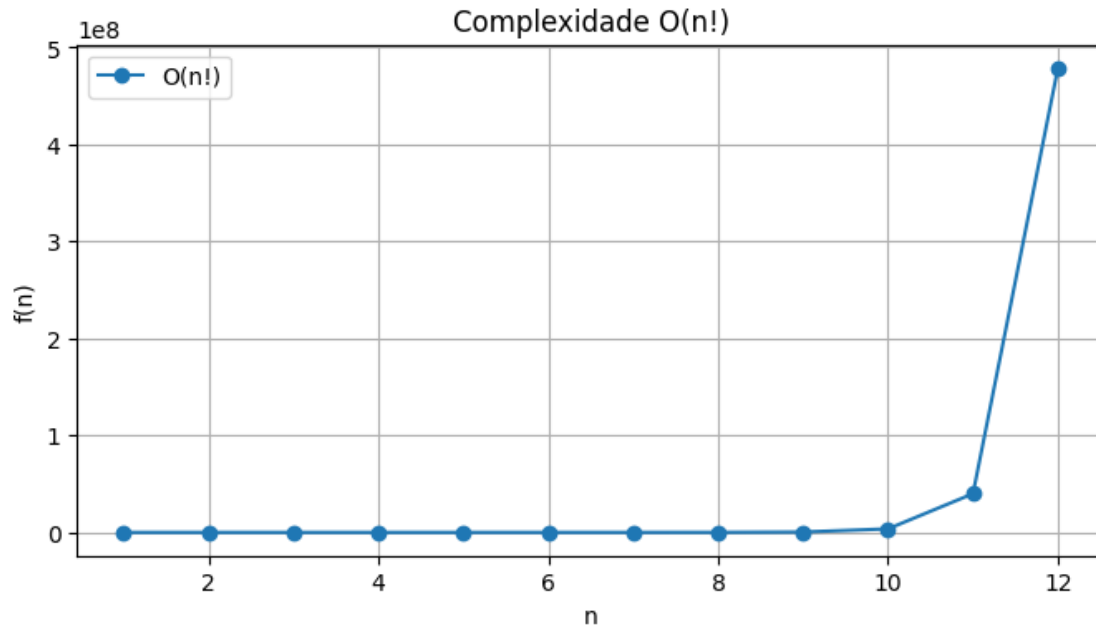
plt.figure(figsize=(8, 4))
plt.plot(ns, o2n, marker="o", label=" $O(2^n)$ ")
plt.title("Complexidade  $O(2^n)$ ")
plt.xlabel("n")
plt.ylabel("f(n)")
plt.grid(True)
plt.legend()
plt.show()
```



```
[7]: # Gráfico individual: O(n!)
import math
import matplotlib.pyplot as plt

ns = list(range(1, 13))
onf = [math.factorial(n) for n in ns]

plt.figure(figsize=(8, 4))
plt.plot(ns, onf, marker="o", label="O(n!)")
plt.title("Complexidade O(n!)")
plt.xlabel("n")
plt.ylabel("f(n)")
plt.grid(True)
plt.legend()
plt.show()
```



6 Gráfico comparativo geral

O gráfico abaixo junta as principais funções de crescimento. Para manter a visualização legível, usamos escala logarítmica no eixo vertical.

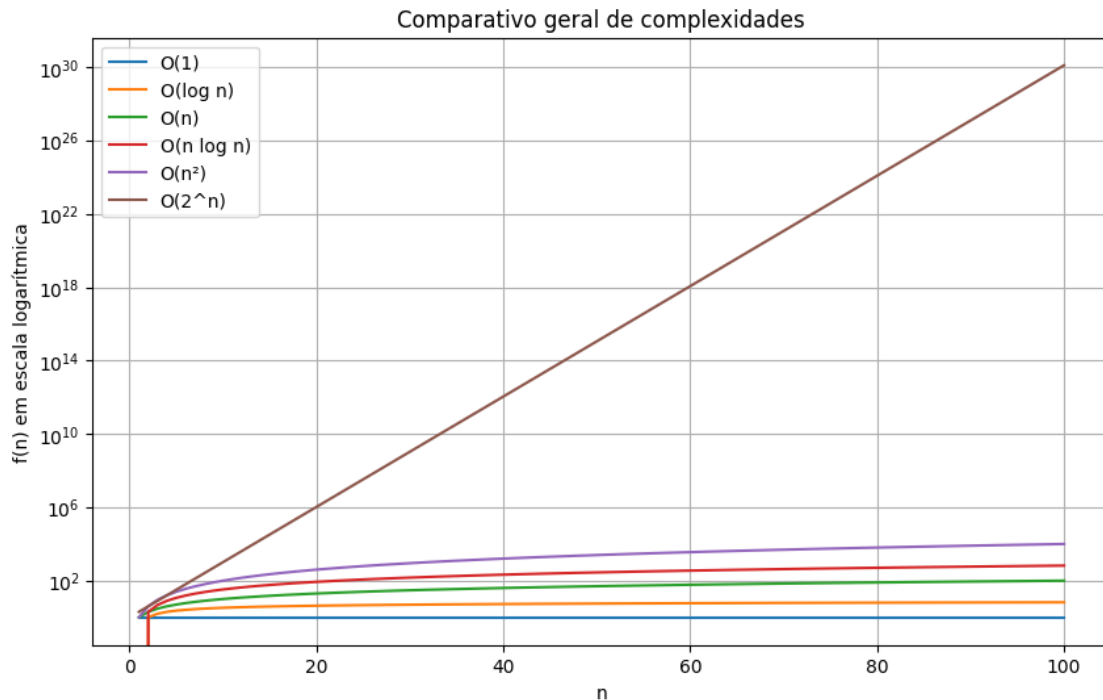
```
[8]: # Gráfico comparativo geral
import math
import matplotlib.pyplot as plt

ns = list(range(1, 101))
comparativo = {
    "O(1)": [1 for n in ns],
    "O(log n)": [math.log2(n) for n in ns],
    "O(n)": [n for n in ns],
    "O(n log n)": [n * math.log2(n) for n in ns],
    "O(n²)": [n**2 for n in ns],
    "O(2^n)": [2**n for n in ns],
}

plt.figure(figsize=(10, 6))
for nome, valores in comparativo.items():
    plt.plot(ns, valores, label=nome)

plt.yscale("log")
plt.title("Comparativo geral de complexidades")
```

```
plt.xlabel("n")
plt.ylabel("f(n) em escala logarítmica")
plt.grid(True)
plt.legend()
plt.show()
```



7 Exemplo em C: acesso direto a vetor — $O(1)$

O acesso direto a uma posição de vetor é considerado $O(1)$, pois o programa calcula diretamente o endereço da posição desejada.

Em C, se temos um vetor `int vetor[5]`, acessar `vetor[2]` não exige percorrer os elementos anteriores. O acesso é direto.

```
#include <stdio.h>

int main() {
    int vetor[5] = {10, 20, 30, 40, 50};

    printf("Elemento na posição 2: %d\n", vetor[2]);

    return 0;
}
```

Mesmo se o vetor tivesse 5, 500 ou 5 milhões de posições, acessar uma posição específica continua

sendo uma operação de custo constante, desde que o índice seja válido.

8 Exemplo em C: busca linear — $O(n)$

A busca linear percorre o vetor elemento por elemento até encontrar o valor desejado ou até chegar ao final.

No pior caso:

- o valor está na última posição; ou
- o valor não está no vetor.

Nesses casos, o algoritmo precisa verificar todos os elementos.

```
#include <stdio.h>

int buscaLinear(int vetor[], int tamanho, int alvo) {
    for (int i = 0; i < tamanho; i++) {
        if (vetor[i] == alvo) {
            return i;
        }
    }

    return -1;
}

int main() {
    int dados[] = {2, 4, 6, 8, 10, 12};
    int tamanho = 6;
    int alvo = 10;

    int posicao = buscaLinear(dados, tamanho, alvo);

    if (posicao != -1) {
        printf("Valor encontrado na posição %d\n", posicao);
    } else {
        printf("Valor não encontrado\n");
    }

    return 0;
}
```

Análise:

- melhor caso: o elemento está na primeira posição, custo $O(1)$;
- pior caso: o elemento está no final ou não existe, custo $O(n)$;
- caso médio: aproximadamente metade do vetor é percorrida, mas a complexidade continua $O(n)$.

9 Exemplo em C: busca binária — $O(\log n)$

A busca binária só funciona corretamente em um vetor **ordenado**.

A ideia é comparar o alvo com o elemento do meio:

- se o alvo for igual, a busca termina;
- se o alvo for maior, descarta-se a metade esquerda;
- se o alvo for menor, descarta-se a metade direita.

```
#include <stdio.h>

int buscaBinaria(int vetor[], int tamanho, int alvo) {
    int inicio = 0;
    int fim = tamanho - 1;

    while (inicio <= fim) {
        int meio = inicio + (fim - inicio) / 2;

        if (vetor[meio] == alvo) {
            return meio;
        } else if (vetor[meio] < alvo) {
            inicio = meio + 1;
        } else {
            fim = meio - 1;
        }
    }

    return -1;
}

int main() {
    int dados[] = {3, 6, 9, 12, 15, 18, 21, 24};
    int tamanho = 8;
    int alvo = 18;

    int posicao = buscaBinaria(dados, tamanho, alvo);

    if (posicao != -1) {
        printf("Valor encontrado na posição %d\n", posicao);
    } else {
        printf("Valor não encontrado\n");
    }

    return 0;
}
```

A busca binária é $O(\log n)$ porque, a cada repetição, o problema fica aproximadamente pela metade.

10 Exemplo em C: Bubble Sort — $O(n^2)$

O Bubble Sort compara elementos vizinhos e troca quando estão fora de ordem.

Ele é simples para fins didáticos, mas não é eficiente para grandes quantidades de dados.

```
#include <stdio.h>

void bubbleSort(int vetor[], int tamanho) {
    int trocou;

    for (int i = 0; i < tamanho - 1; i++) {
        trocou = 0;

        for (int j = 0; j < tamanho - 1 - i; j++) {
            if (vetor[j] > vetor[j + 1]) {
                int temp = vetor[j];
                vetor[j] = vetor[j + 1];
                vetor[j + 1] = temp;
                trocou = 1;
            }
        }

        if (!trocou) {
            break;
        }
    }
}

void imprimirVetor(int vetor[], int tamanho) {
    for (int i = 0; i < tamanho; i++) {
        printf("%d ", vetor[i]);
    }
    printf("\n");
}

int main() {
    int dados[] = {8, 3, 5, 1, 9, 2};
    int tamanho = 6;

    printf("Antes: ");
    imprimirVetor(dados, tamanho);

    bubbleSort(dados, tamanho);

    printf("Depois: ");
    imprimirVetor(dados, tamanho);

    return 0;
}
```

```
}
```

Análise:

- o laço externo executa aproximadamente n vezes;
- o laço interno também executa aproximadamente n vezes;
- por isso, o crescimento é proporcional a $n * n$, ou seja, $O(n^2)$.

11 Exemplo em C: memória estática e dinâmica

Em C, a complexidade de espaço também deve ser observada.

11.1 Vetor estático

```
#include <stdio.h>

int main() {
    int vetor[100];

    for (int i = 0; i < 100; i++) {
        vetor[i] = i;
    }

    printf("Primeiro elemento: %d\n", vetor[0]);
    printf("Último elemento: %d\n", vetor[99]);

    return 0;
}
```

Nesse caso, o tamanho do vetor já é definido no código.

11.2 Vetor dinâmico com malloc()

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n;

    printf("Digite o tamanho do vetor: ");
    scanf("%d", &n);

    int *vetor = malloc(n * sizeof(int));

    if (vetor == NULL) {
        printf("Erro ao alocar memória\n");
        return 1;
    }
}
```

```

for (int i = 0; i < n; i++) {
    vetor[i] = i * 2;
}

printf("Primeiro elemento: %d\n", vetor[0]);
printf("Último elemento: %d\n", vetor[n - 1]);

free(vetor);

return 0;
}

```

Nesse exemplo, o espaço usado pelo vetor cresce conforme n . Portanto, o consumo de memória é $O(n)$.

Em C, sempre que usar `malloc()`, é necessário liberar a memória com `free()` quando ela não for mais necessária.

12 Medição empírica dos exemplos

Os algoritmos principais foram apresentados em C. Porém, para gerar gráficos dentro do notebook, usaremos Python para simular o comportamento e visualizar as tendências.

A ideia não é substituir C por Python, mas usar Python como ferramenta de apoio visual.

```

[1]: # Funções em Python usadas apenas para gerar gráficos empíricos no notebook
import timeit
import random

def busca_linear_py(arr, alvo):
    for i, x in enumerate(arr):
        if x == alvo:
            return i
    return -1

def busca_binaria_py(arr, alvo):
    inicio = 0
    fim = len(arr) - 1

    while inicio <= fim:
        meio = inicio + (fim - inicio) // 2

        if arr[meio] == alvo:
            return meio
        elif arr[meio] < alvo:
            inicio = meio + 1
        else:

```

```

        fim = meio - 1

    return -1

def bubble_sort_py(arr):
    a = arr[:]
    n = len(a)

    for i in range(n - 1):
        trocou = False

        for j in range(0, n - 1 - i):
            if a[j] > a[j + 1]:
                a[j], a[j + 1] = a[j + 1], a[j]
                trocou = True

        if not trocou:
            break

    return a

```

```

[2]: # Comparação empírica: busca linear  $O(n)$  vs busca binária  $O(\log n)$ 
import matplotlib.pyplot as plt

def medir_buscas(tamanhos, repeticoes=3):
    tempos_linear = []
    tempos_binaria = []

    for n in tamanhos:
        arr = list(range(n))
        alvo = n - 1

        t_linear = min(timeit.repeat(lambda: busca_linear_py(arr, alvo),
↪number=1, repeat=repeticoes))
        t_binaria = min(timeit.repeat(lambda: busca_binaria_py(arr, alvo),
↪number=100, repeat=repeticoes)) / 100

        tempos_linear.append(t_linear)
        tempos_binaria.append(t_binaria)

    return tempos_linear, tempos_binaria

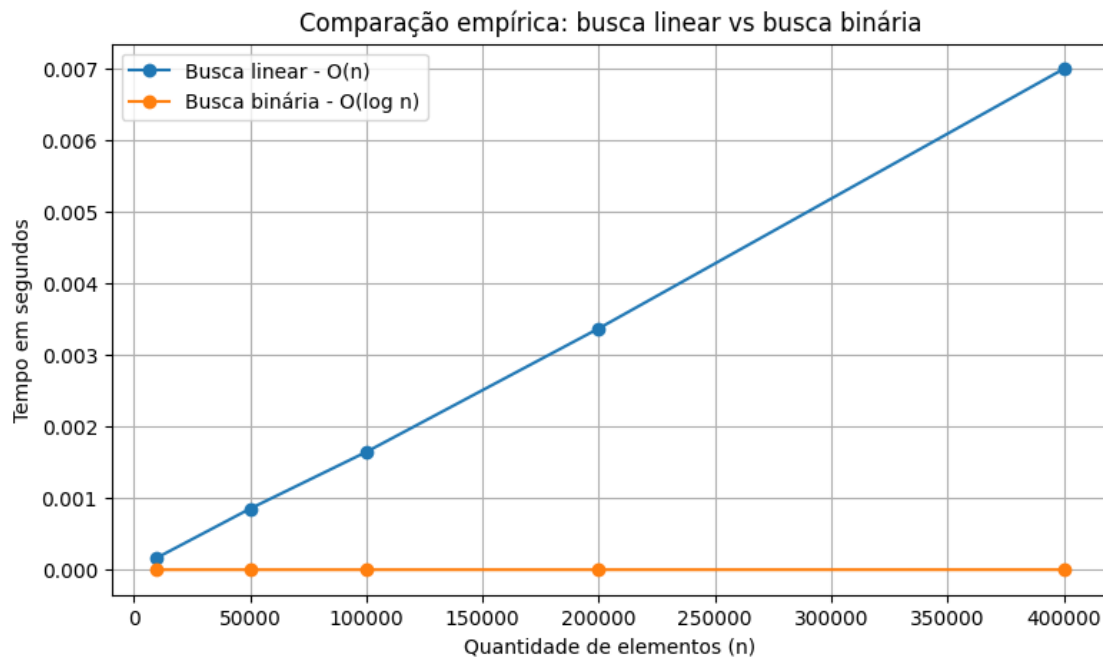
tamanhos_busca = [10_000, 50_000, 100_000, 200_000, 400_000]
res_linear, res_binaria = medir_buscas(tamanhos_busca)

```

```

plt.figure(figsize=(9, 5))
plt.plot(tamANHos_busca, res_linear, marker="o", label="Busca linear -  $O(n)$ ")
plt.plot(tamANHos_busca, res_binaria, marker="o", label="Busca binária -  $O(\log n)$ ")
plt.title("Comparação empírica: busca linear vs busca binária")
plt.xlabel("Quantidade de elementos (n)")
plt.ylabel("Tempo em segundos")
plt.grid(True)
plt.legend()
plt.show()

```



```

[11]: # Comparação empírica: Bubble Sort  $O(n^2)$  vs sorted()  $O(n \log n)$ 
import matplotlib.pyplot as plt

def medir_ordenacao(tamANHos, repeticoes=3):
    tempos_bubble = []
    tempos_sorted = []

    for n in tamANHos:
        base = [random.randint(0, 10_000) for _ in range(n)]

        t_bubble = min(timeit.repeat(lambda: bubble_sort_py(base), number=1,
repeat=repeticoes))
        t_sorted = min(timeit.repeat(lambda: sorted(base), number=1,
repeat=repeticoes))

```

```

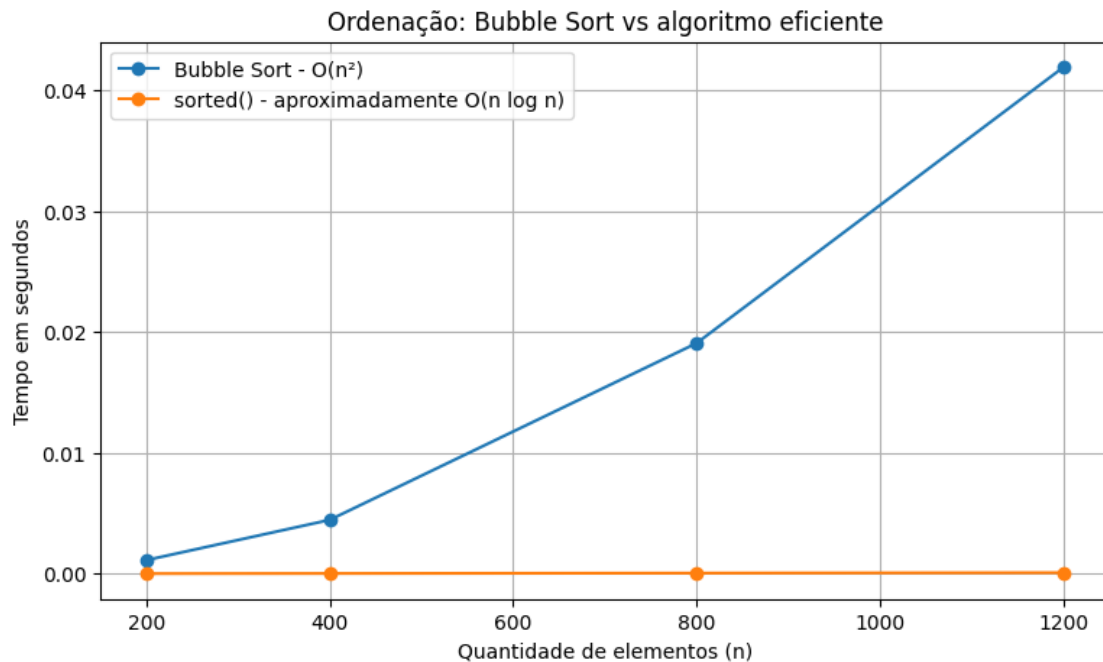
    tempos_bubble.append(t_bubble)
    tempos_sorted.append(t_sorted)

    return tempos_bubble, tempos_sorted

tamanhos_ord = [200, 400, 800, 1200]
tb, ts = medir_ordenacao(tamanhos_ord)

plt.figure(figsize=(9, 5))
plt.plot(tamanhos_ord, tb, marker="o", label="Bubble Sort -  $O(n^2)$ ")
plt.plot(tamanhos_ord, ts, marker="o", label="sorted() - aproximadamente  $O(n \log n)$ ")
plt.title("Ordenação: Bubble Sort vs algoritmo eficiente")
plt.xlabel("Quantidade de elementos (n)")
plt.ylabel("Tempo em segundos")
plt.grid(True)
plt.legend()
plt.show()

```



13 Como analisar laços em C

Ao analisar um algoritmo em C, observe principalmente os laços.

13.1 Um laço simples

```
for (int i = 0; i < n; i++) {  
    printf("%d\n", i);  
}
```

Esse trecho executa proporcionalmente a n , portanto é $O(n)$.

13.2 Dois laços separados

```
for (int i = 0; i < n; i++) {  
    printf("%d\n", i);  
}
```

```
for (int j = 0; j < n; j++) {  
    printf("%d\n", j);  
}
```

O custo é $n + n$, ou seja, $2n$. Como ignoramos constantes, a complexidade é $O(n)$.

13.3 Laços aninhados

```
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < n; j++) {  
        printf("%d %d\n", i, j);  
    }  
}
```

Aqui, para cada valor de i , o laço de j executa n vezes. O custo é $n * n$, portanto $O(n^2)$.

14 Exercícios sem código

1. Um algoritmo acessa diretamente a posição `vetor[10]`. Qual é a complexidade?
2. Um algoritmo percorre um vetor do início ao fim procurando um valor. Qual é a complexidade no pior caso?
3. Um algoritmo possui dois laços `for` separados, cada um indo de 0 até $n - 1$. A complexidade é $O(n)$ ou $O(n^2)$? Justifique.
4. Um algoritmo possui um laço dentro de outro, ambos indo de 0 até $n - 1$. Qual é a complexidade?
5. Por que a busca binária precisa de um vetor ordenado?
6. Explique por que o Bubble Sort é $O(n^2)$.
7. Em C, por que é importante usar `free()` depois de `malloc()`?
8. Um algoritmo usa um vetor auxiliar com n posições. Qual é a complexidade de espaço?

15 Exercícios com código em C

15.1 Exercício 1

Implemente uma função em C chamada `maiorValor` que receba um vetor de inteiros e retorne o maior valor encontrado.

Depois, responda:

- qual é a complexidade de tempo?
- qual é a complexidade de espaço?

15.2 Exercício 2

Implemente uma busca linear em C para procurar o nome de um aluno em um vetor de strings.

Depois, responda:

- qual é o melhor caso?
- qual é o pior caso?
- qual é a complexidade Big-O?

15.3 Exercício 3

Implemente o Bubble Sort para ordenar um vetor de notas.

Depois, responda:

- por que existe laço aninhado?
- quantas comparações aproximadamente ocorrem quando n cresce?
- por que a complexidade é $O(n^2)$?

15.4 Exercício 4

Crie um vetor dinâmico usando `malloc()` para armazenar n números inteiros digitados pelo usuário.

Depois:

- preencha o vetor;
- exiba os valores;
- libere a memória com `free()`;
- explique a complexidade de espaço.

16 Dicas finais para análise assintótica

- Ignore constantes: $O(2n)$ vira $O(n)$.
- Ignore termos menores: $O(n^2 + n + 10)$ vira $O(n^2)$.
- Um laço simples geralmente indica $O(n)$.
- Laços aninhados geralmente indicam $O(n^2)$, $O(n^3)$, etc.
- Algoritmos que dividem o problema pela metade costumam ser $O(\log n)$.
- Algoritmos de ordenação eficientes geralmente ficam próximos de $O(n \log n)$.
- Em C, observe também o uso de memória: vetores, ponteiros, `malloc()`, `free()` e estruturas auxiliares.